CHAPTER 3 Random Variables and Probability Distributions

§ 3-1 Introduction

Figure 3-1 Continuous iteration between model and physical system.

Figure 3-2 Noise variables affect the transformation of inputs to
outputs.

Figure 3-3 A closer examination of the system identifies deviations from
the model.

§ 3-2 Random Variables

EXERCISES FOR § 3-2

§ 3-3 Probability

EXAMPLE 3-1 Hospital Emergency Visits

EXERCISES FOR § 3-3

§ 3-4 Continuous Random Variables

§ 3-4.1 Probability Density Function

Figure 3-5 Density function of a loading on a long, thin beam.

Figure 3-6 Probability determined from the area under f(x).

Figure 3-7 A histogram approximates a probability density function. The
area of each bar equals the relative frequency of the interval. The area
under f(x) over any interval equals the probability of the interval.

EXAMPLE 3-2 Current in a Wire

Figure 3-8 Probability density function for Example 3-2.

```
# --- 1. 3-4.1 Probability Density Function (P.D.F.) ---

# EXAMPLE 3-2: Current in a Wire (Uniform Distribution)
# Probability x < 10 and 5 < x < 10
p_less_10 <- punif(10, min = 0, max = 20)
p_5_to_10 <- punif(10, 0, 20) - punif(5, 0, 20)

# Plotting Uniform PDF
x_vals <- seq(-5, 25, length.out = 1000)
y_vals <- dunif(x_vals, 0, 20)

plot(x_vals, y_vals, type = "l", lwd = 2, col = "blue",
     main = "Uniform(0, 20) P.D.F.",
     xlab = "x", ylab = "f(x)", ylim = c(0, 0.06))

# Highlight area for P(0 < X < 10)
polygon(c(0, seq(0, 10, len=100), 10),
        c(0, dunif(seq(0, 10, len=100), 0, 20), 0),
        col = rgb(0, 1, 0, 0.3), border = NA)

# EXAMPLE 3-3 Flaw on a Magnetic Disk

# Figure 3-9 Probability density function for Example 3-3.

# EXAMPLE 3-3: Flaw on a Magnetic Disk (Exponential Distribution)
rate_val <- 1/2000
p_gt_1000 <- 1 - pexp(1000, rate = rate_val)
p_1000_to_2000 <- pexp(2000, rate_val) - pexp(1000, rate_val)

# Plotting Exponential PDF
x_exp <- seq(0, 8000, length.out = 1000)
y_exp <- dexp(x_exp, rate = rate_val)

plot(x_exp, y_exp, type = "l", lwd = 2, col = "darkred",
     main = "Exponential(rate = 1/2000) P.D.F.",
     xlab = "x", ylab = "f(x)")
```

```
# Shading area for P(X > 1000)
x_shade <- seq(1000, 8000, length.out = 100)
polygon(c(1000, x_shade, 8000),
        c(0, dexp(x_shade, rate_val), 0),
        col = rgb(1, 0.5, 0, 0.3), border = NA)

# § 3-4.2 Cumulative Distribution Function

# EXAMPLE 3-4 Flaw on a Magnetic Disk Distribution Function

# Figure 3-10 Cumulative distribution function for Example 3-4.

# --- 2. 3-4.2 Cumulative Distribution Function (C.D.F.) ---

# EXAMPLE 3-4: Exponential C.D.F.
plot(x_exp, pexp(x_exp, rate_val), type = "l", lwd = 2,
     main = "Exponential(1/2000) C.D.F.",
     xlab = "x", ylab = "F(x)", col = "darkgreen")
abline(h = c(0, 1), lty = 2, col = "gray") # Reference lines

lines(c(0, 2000), c(0, 1), col = 2)

# § 3-4.3 Mean and Variance

# EXAMPLE 3-5 Current in a Wire: Mean

# EXAMPLE 3-6 Current in a Wire: Variance

# --- 3. 3-4.3 Mean and Variance ---

# EXAMPLE 3-5 & 3-6: Uniform Stats (min = 0, max = 20)
u_mean <- (0 + 20) / 2
u_var  <- (20 - 0)^2 / 12

cat("Uniform Mean:", u_mean, "\nUniform Variance:", u_var)

# EXERCISES FOR § 3-4
```

```
# § 3-5 Important Continuous Distributions

# § 3-5.1 Normal Distribution

# Figure 3-11 Normal probability density functions for selected
# values of the parameters μ and σ².

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(HH)

# Save original parameters and set a cleaner margins/layout
old.par <- par(no.readonly = TRUE)
par(mfrow = c(1, 1), oma = c(0, 1, 0, 4), mar = c(5, 4, 4, 2) + 0.1)

# --- 2. Figure 3-11: Normal PDFs comparison ---
norm.setup(xlim = c(-4, 18), mean = 10, se = 1)
norm.curve(mean = 5, se = 1, col = 1)
norm.curve(mean = 15, se = 1, col = 2, add = TRUE)
norm.curve(mean = 5, se = 3, col = 3, add = TRUE)
legend("topright",
       legend = c("μ = 5, σ = 1", "μ = 15, σ = 1", "μ = 5, σ = 3"),
       col = 1:3, lty = 1)

# EXAMPLE 3-7 Current in a Wire: Normal Distribution

# Figure 3-12 Probability that X > 13 for a normal random variable with μ
# = 10 and σ² = 4 in Example 3-7.

# --- 3. EXAMPLE 3-7: Current in a Wire ---
# Prob X > 13 (μ = 10, σ = 2)
norm.setup(xlim = c(4, 16), mean = 10, se = 2)
norm.curve(10, 2, critical.values = 13)

# Figure 3-13 Probabilities associated with a normal distribution.
```

```
# --- 4. Empirical Rule Visuals (1, 2, 3σ) ---
norm.setup(xlim = c(-3, 3))
norm.curve(0, 1, c(-1, 1), shade = "inside")
mtext(paste(1, "σ"), side = 3)

norm.setup(xlim = c(-3, 3))
norm.curve(0, 1, c(-2, 2), shade = "inside")
mtext(paste(2, "σ"), side = 3)

norm.setup(xlim = c(-3, 3))
norm.curve(0, 1, c(-3, 3), shade = "inside")
mtext(paste(3, "σ"), side = 3)

# Calculated probabilities for the Empirical Rule
empirical_probs <- pnorm(1:3) - pnorm(-(1:3))
print(empirical_probs)

# EXAMPLE 3-8 Standard Normal Distribution

# Figure 3-14 Standard normal probability density function.

# --- 5. EXAMPLE 3-8: Standard Normal Distribution ---
# Prob X < 1.5 (μ = 1, σ = 1)

norm.setup(xlim = c(-3, 3))
norm.curve(0, 1, 1.5, shade = "left")

pnorm(c(0, .01, .02, .03))

pnorm(c(1.5, 1.51, 1.52, 1.53))

# EXAMPLE 3-9 Normal Distribution Probabilities

# Figure 3-15 Graphical displays for Example 3-9.
```

```
# --- 6. EXAMPLE 3-9: Standard Normal Probabilities ---
# Combined logic for some of Figure 3-15
norm.setup(xlim = c(-3, 3))
norm.curve(0, 1, 1.26, shade = "left")  # P(Z < 1.26)

norm.setup(xlim = c(-3, 3))
norm.curve(0, 1, c(-1.25, 0.37), shade = "inside") # P(-1.25 < Z < 0.37)

# Finding Z-score for 95th percentile
z_95 <- qnorm(0.95)
norm.setup(xlim = c(-3, 3))
norm.curve(0, 1, z_95, shade = "left")

# EXAMPLE 3-10 Current in a Wire: Normal Distribution Probability

# Figure 3-16 Standardizing a normal random variable.

# EXAMPLE 3-11 Current in a Wire: Normal Distribution Probability

# Figure 3-17 Determining the value of x to meet a specified probability,
# Example 3-11.

# --- 7. EXAMPLE 3-11: Inverse Normal ---
# Find x such that P(X < x) = 0.98 (μ=10, σ=2)
x_val <- qnorm(0.98, mean = 10, sd = 2)
norm.setup(xlim = c(4, 16), mean = 10, se = 2)
norm.curve(10, 2, x_val, shade = "left")

# EXAMPLE 3-12 Voltage of Noise

norm.setup(xlim = c(-3, 3)*.45, se = .45)
norm.curve(0, .45, .9)

# Figure 3-18 Determining the value of x to meet a specified probability,
# Example 3-12.

norm.setup(xlim = c(-3, 3)*.45, se = .45)
```

```
norm.curve(0, .45, c(-1, 1)*.45*qnorm(1 - .005), shade = "inside")

norm.setup(xlim = 1.8 + c(-3, 3)*.45, mean = 1.8, se = .45)

norm.curve(1.8, .45, .9, shade = "left")

# EXAMPLE 3-13 Diameter of a Shaft

# Figure 3-19 Distribution for Example 3-13.

# --- 8. EXAMPLE 3-13: Shaft Diameter ---
# Centered at .2508, showing spec limits at .2500 +/- .0015
mu_shaft <- 0.2508
sd_shaft <- 0.0005
norm.setup(xlim = c(0.2485, 0.2525), mean = mu_shaft, se = sd_shaft)
norm.curve(mu_shaft, sd_shaft, c(0.2485, 0.2515), shade = "inside")

# Reset plotting parameters
par(old.par)

# § 3-5.2 Lognormal Distribution

# Figure 3-20 Lognormal probability density functions with θ = 0 for
# selected values of ω².

# --- 1. Setup ---
# Define parameters
sd_vals <- c(0.5, 1, 1.5)
colors  <- c("black", "red", "blue")
x_max   <- exp(1.5^2/2) * (1 + 3 * sqrt(exp(1.5^2) - 1))
x       <- seq(0, x_max, length.out = 1000)

# Create base plot with the first distribution (sdlog = 0.5)
plot(x, dlnorm(x, meanlog = 0, sdlog = sd_vals[1]),
     type = "l", col = colors[1], lwd = 2,
     main = "Lognormal PDFs (theta = 0, meanlog = 0)",
     xlab = "x", ylab = "f(x)")
```

```
# Add remaining lines using a loop
for(i in 2:3) {
  lines(x, dlnorm(x, meanlog = 0, sdlog = sd_vals[i]),
        col = colors[i], lwd = 2)
}

# Add a legend
legend("topright", legend = paste("sdlog =", sd_vals),
       col = colors, lwd = 2)

# EXAMPLE 3-14 Lifetime of a Laser

# --- 2. EXAMPLE 3-14: Lifetime of a Laser ---
m <- 10; s <- 1.5

# Probability lifetime exceeds 10,000
prob_exceed <- plnorm(10000, m, s, lower.tail = FALSE)

# 99th percentile (value exceeded by 99% of population)
q_99th <- qlnorm(0.01, m, s)

# Mean and Variance
mean_val <- exp(m + s^2/2)
var_val  <- exp(2*m + s^2) * (exp(s^2) - 1)
sd_val   <- sqrt(var_val)

# Results Printout
cat("Prob > 10k:", prob_exceed, "\n1st Percentile:", q_99th,
    "\nMean:", mean_val, "\nStd Dev:", sd_val)

# § 3-5.3 Gamma Distribution

gamma(1:10)
gamma(1/2); sqrt(pi)

# Figure 3-21 Gamma probability density functions for selected values of λ
# and r.
```

```
# --- 1. Setup ---
# Function to plot multiple curves to avoid repetition
plot_dist_group <- function(x_vals, df_range, title, is_chisq = TRUE) {
  x_max <- max(x_vals)
  x_seq <- seq(0, x_max, length.out = 1001)

  # Initialize empty plot
  plot(NULL, xlim = c(0, x_max), ylim = c(0, 0.5),
       main = title, xlab = "x", ylab = "f(x)")

  # Draw lines
  for (i in seq_along(df_range)) {
    y <- if(is_chisq) dchisq(x_seq, df_range[i]) else
         dgamma(x_seq, shape = df_range[i][1], rate = df_range[i][2])
    lines(x_seq, y, col = i)
  }
}

# --- 2. Gamma Plots (Figure 3-21) ---
u1_max <- max(c(1, 1/2, 1/3.75) * (c(1, 8.3, 7.5) +
          3*sqrt(c(1, 8.3, 7.5))))
x_gamma <- seq(0, u1_max, len = 1001)

plot(x_gamma, dgamma(x_gamma, 1, 1), type = "l",
     main = "Gamma P.D.F. Comparison", xlab = "x", ylab = "f(x)")
lines(x_gamma, dgamma(x_gamma, 8.3, 2), col = 2)
lines(x_gamma, dgamma(x_gamma, 7.5, 3.75), col = 3)
legend("topright", legend = c("G(1, 1)", "G(8.3, 2)", "G(7.5, 3.75)"),
       col = 1:3, lty = 1)

# --- 3. Chi-Squared Plots (Automated) ---
# Group 1: k = 1 to 10
u2 <- 1:10 + 3*sqrt(2*(1:10))
plot_dist_group(u2, 1:10,
                expression({chi^2}[k] ~~"p.d.f. for k = 1, 2, ..., 10"))
```

```
# Group 2: k = 11 to 20
u3 <- 11:20 + 3*sqrt(2*(11:20))
plot_dist_group(u3, 11:20,
                expression({chi^2}[k] ~~"p.d.f. for k = 11, 12, ..., 20"))

# Group 3: k = 21 to 30
u4 <- 21:30 + 3*sqrt(2*(21:30))
plot_dist_group(u4, 21:30,
                expression({chi^2}[k] ~~"p.d.f. for k = 21, 22, ..., 30"))

# Group 4: k = 10, 20, ..., 100
u5 <- 10 * 1:10 + 3*sqrt(2 * 10 * 1:10)
plot_dist_group(u5, seq(10, 100, by = 10),
                expression({chi^2}[k] ~~"p.d.f. for k = 10, 20, ..., 100"))

# § 3-5.4 Weibull Distribution

# --- 1. Setup ---
# Define parameters
shapes <- c(1, 2, 6.5)
scales <- c(1, 3.4, 4.5)

# Calculate upper bounds (Mean + 3*SD) for plotting
u6 <- scales * (gamma(1 + 1/shapes) +
      3 * sqrt(gamma(1 + 2/shapes) - gamma(1 + 1/shapes)^2))

x_vals <- seq(0, max(u6), length.out = 1001)

# --- 2. Plotting ---
# Plot first density (Shape = 1, Scale = 1)
plot(x_vals, dweibull(x_vals, shape = shapes[1], scale = scales[1]),
     type = "l", col = 1, lwd = 2,
     main = "Weibull Distributions Comparison",
     xlab = expression(x), ylab = expression(f(x)))

# Add second density (Shape = 2, Scale = 3.4)
lines(x_vals, dweibull(x_vals, shapes[2], scales[2]), col = 2, lwd = 2)
```

```
# Add third density (Shape = 6.5, Scale = 4.5)
lines(x_vals, dweibull(x_vals, shapes[3], scales[3]), col = 3, lwd = 2)

# Add a legend for clarity
legend("topright",
       legend = paste0("Shape = ", shapes, ", Scale = ", scales),
       col = 1:3, lty = 1, lwd = 2)

# --- 3. EXAMPLE 3-15 Lifetime of a Bearing ---
# Expected lifetime (Mean)
mean_life <- 5000 * gamma(1 + 1/0.5)
print(paste("Mean Lifetime:", mean_life))
# Probability lifetime exceeds 6000
prob_6000 <- pweibull(6000, shape = 0.5, scale = 5000, lower.tail = FALSE)
print(paste("P(X > 6000):", prob_6000))

# § 3-5.5 Beta Distribution

# Figure 3-23 Beta probability density functions for selected values of the
# parameters alpha and beta.

# --- 1. Setup ---
# Define common x-axis values
x <- seq(0.001, 0.999, length.out = 1000)

# --- 2. Plotting ---
# Initialize plot with the first distribution (shape1 = 5, shape2 = 1)
plot(x, dbeta(x, 5, 1), type = "l", col = "red", lwd = 2,
     main = "Beta Probability Density Functions",
     xlab = expression(x), ylab = expression(f(x)),
     ylim = c(0, 3.5)) # Set ylim to fit all curves

# Add the other distributions
lines(x, dbeta(x, 0.5, 0.5), col = "black",  lwd = 2)
lines(x, dbeta(x, 1, 3),     col = "green",  lwd = 2)
lines(x, dbeta(x, 2, 2),     col = "blue",   lwd = 2)
lines(x, dbeta(x, 2, 5),     col = "orange", lwd = 2)
```

```
# Add a legend for clarity
legend("top",
       legend = c("beta(5, 1)", "beta(0.5, 0.5)", "beta(1, 3)", "beta(2,
2)", "beta(2, 5)"),
       col = c("red", "black", "green", "blue", "orange"), lty = 1,
       lwd = 2, cex = 0.8)

# --- 3. Examples ---
# EXAMPLE 3-16: P(X > 0.7)
prob_gt_07 <- pbeta(0.7, 2.5, 1, lower.tail = FALSE)
print(paste("Probability > 0.7:", round(prob_gt_07, 4)))

# EXAMPLE 3-17: Mean and Variance
alpha <- 2.5; beta <- 1
mean_val <- alpha / (alpha + beta)
var_val <- (alpha * beta) / ((alpha + beta)^2 * (alpha + beta + 1))

cat("Mean:", mean_val, "\nVariance:", var_val, "\n")

# EXERCISES FOR § 3-5
```

```
# § 3-6 Probability Plots

# § 3-6.1 Normal Probability Plots

# EXAMPLE 3-18 Service Life of a Battery

# Figure 3-24 Normal probability plot for the battery life.

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(car, ggplot2, e1071)

# --- 2. 3-6.1 Normal Probability Plots (EXAMPLE 3-18) ---
SvcL <- c(176, 191, 214, 220, 205, 192, 201, 190, 183, 185)
SvcLBat <- data.frame(ObsNo = 1:length(SvcL), SvcL = SvcL)

# Figure 3-25 Normal probability plot obtained from standardized normal
# scores.

summary(SvcLBat)
qqPlot(SvcLBat$SvcL, main = "Normal Probability Plot: Battery Life")
shapiro.test(SvcLBat$SvcL)

# § 3-6.2 Other Probability Plots

# Interpeting a Probability Plot

# Table 3-1 Crack Length (mm) for an Aluminum Alloy

# --- 3. 3-6.2 Other Probability Plots (Table 3-1 Crack Length) ---
CrackLn <- c(81, 249, 117, 227, 134, 98, 135, 149, 225, 59, 291, 223,
             127, 185, 181, 101, 205, 115, 240, 151, 98, 80, 198, 161, 240,
             118, 117, 342, 197, 146, 158, 82, 83, 98, 104, 197, 64, 34,
65,
             100, 139, 137, 342, 144, 215, 249, 149, 185, 151, 200)

CrackLnAlAly <- data.frame(ObsNo = 1:length(CrackLn), CrackLn = CrackLn)
```

```
# Figure 3-26 Normal probability plot for the crack-length data in Table
# 3-1.

# Normal Plot for Crack Length
qqPlot(CrackLnAlAly$CrackLn, main = "Normal QQ Plot: Crack Length")
shapiro.test(CrackLnAlAly$CrackLn)

# Figure 3-27 Lognormal probability plot for the crack-length data in Table
# 3-1.

# --- 4. Lognormal Probability Plot ---
LogCrackLn <- log(CrackLn)
qqPlot(LogCrackLn, main="Lognormal QQ Plot: Crack Length")
shapiro.test(LogCrackLn)

# Figure 3-28 Weibull probability plot for the crack-length data in Table
# 3-1.

# --- 5. Weibull Probability Plot (Figure 3-28) ---
# A Weibull distribution becomes linear when log-transformed data is
# plotted against Gumbel (Extreme Value) quantiles.
n <- length(LogCrackLn)
p <- (1:n - 0.5) / n
gumbel_quantiles <- log(-log(1 - p))

plot(gumbel_quantiles, sort(LogCrackLn),
     main = "Weibull Probability Plot",
     xlab = "Theoretical Gumbel Quantiles",
     ylab = "log(Sample Crack Length)",
     pch = 19, col = "blue")
abline(lm(sort(LogCrackLn) ~ gumbel_quantiles), col = "red")

# EXERCISES FOR § 3-6
```

```
# § 3-7 Discrete Random Variables

# EXAMPLE 3-19 Voice Network

# EXAMPLE 3-20 Semiconductor Wafer Contamination

# § 3-7.1 Probability Mass Function

# EXAMPLE 3-21 Bit Transmission Errors

# Figure 3-29 Probability distribution for X in Example 3-21.

# --- 1. PMF (Probability Mass Function) ---
# Example 3-21 Data
x_vals <- 0:4
n <- 4
p <- 1/10
pmf_vals <- dbinom(x_vals, size = n, prob = p)

# Figure 3-29: PMF Plot
plot(x_vals, pmf_vals,
     type = "h", lwd = 2, col = "blue",
     main = "Binomial(n = 4; p = 0.1) P.M.F.",
     xlab = "x", ylab = "f(x)",
     ylim = c(0, max(pmf_vals) + 0.1),
     xaxt = "n")
axis(1, at = x_vals)
points(x_vals, pmf_vals, pch = 19, col = "blue")

# Figure 3-30 Loadings at discrete points on a long, thin beam.

# § 3-7.2 Cumulative Distribution Function

# EXAMPLE 3-22 Bit Transmission Errors

# Figure 3-31 Cumulative distribution function for x in Example 3-22.
```

```
# --- 2. CDF (Cumulative Distribution Function) ---
# Example 3-22 & Figure 3-31

# Step function plot
plot(stepfun(x_vals, c(0, pbinom(x_vals, n, p))),
     verticals = FALSE,
     pch = 16,
     col.points = "red",
     main = "Binomial(4; 0.1) C.D.F.",
     xlab = "x", ylab = "F(x)",
     do.points = TRUE)

# Adding the hollow circles for jumps (optional but professional)
points(x_vals, c(0, pbinom(x_vals[1:4], n, p)), pch = 1)

# § 3-7.3 Mean and Variance

# EXAMPLE 3-23 Bit Transmission Errors: Mean and Variance

# --- 3. Mean and Variance ---
# Example 3-23
mu <- n * p
variance <- n * p * (1 - p)

cat("Mean (E[X]):", mu, "\n")
cat("Variance (Var[X]):", variance, "\n")

# EXAMPLE 3-24 Product Revenue

# EXERCISES FOR § 3-7
```

```
# § 3-8 Binomial Distribution

# EXAMPLE 3-25 Bit Transmission Errors

# --- 1. Comparing Binomial and Normal Approximation ---
# P(X = 10) for Binomial vs Normal approximation with continuity correction
binom_val <- dbinom(10, 20, 0.5)
norm_approx <- pnorm(10.5, 10, sqrt(5)) - pnorm(9.5, 10, sqrt(5))

cat("Binomial:", binom_val, "\nNormal Approx:", norm_approx, "\n")

# --- 2. Visualization: Binomial vs. Normal ---
# Plotting Normal Curve vs. Binomial bars
x_vals <- seq(-0.5, 20.5, length.out = 1000)
y_norm <- dnorm(x_vals, 10, sqrt(5))

plot(x_vals, y_norm, type = "l", col = "red", lwd = 2,
     main = "Binomial(20; 0.5) vs. N(10, 5)",
     xlab = "x", ylab = "f(x)")

# Overlay Binomial PMF as vertical lines (type = "h") and dots
points(0:20, dbinom(0:20, 20, 0.5), type = "h", col = "blue")
points(0:20, dbinom(0:20, 20, 0.5), pch = 16, col = "blue")

# --- 3. Comparison of Binomial and Poisson ---
# Set up the plot window
plot(0:10, dbinom(0:10, 10, 0.1), type = "h", lwd = 2,
     main = "Binomial vs. Poisson Comparison",
     xlab = "x", ylab = "Probability", ylim = c(0, 0.4))

# Binomial(10; 0.1) - Black
points(0:10, dbinom(0:10, 10, 0.1), pch = 16)

# Binomial(10, 0.9) - Red
points(0:10, dbinom(0:10, 10, 0.9), type = "h", col = "red")
points(0:10, dbinom(0:10, 10, 0.9), col = "red", pch = 16)
```

```
# Poisson(1) - Green
points(0:10, dpois(0:10, lambda = 1), type = "h", col = "green")
points(0:10, dpois(0:10, lambda = 1), col = "green", pch = 17)

# 10 - Poisson(1) - Blue
points(0:10, dpois(10:0, lambda = 1), type = "h", col = "blue")
points(0:10, dpois(10:0, lambda = 1), col = "blue", pch = 18)

legend("top",
       legend = c("Binomial(10; 0.1)", "Binomial(10; 0.9)", "Poisson(1)",
                  "10 - Poisson(1)"),
       col = c("black", "red", "green", "blue"), pch = c(16, 16, 17, 18),
       horiz = TRUE, cex = 0.7)

# Figure 3-32 Binomial distribution for selected values of n and p.

# EXAMPLE 3-26 Binomial Coefficient

# EXAMPLE 3-27 Organic Solids

# --- 4. Examples 3-26 & 3-27: Practical Applications ---
# EXAMPLE 3-26: Combinations (nCr)
choose(10, 3)
choose(15, 10)
choose(100, 4)

# EXAMPLE 3-27: Organic Solids (n = 18, p = 0.1)
# P(X = 2)
dbinom(2, 18, 0.1)

# P(X >= 4) = 1 - P(X <= 3)
1 - pbinom(3, 18, 0.1)

# P(3 <= X < 7) = P(X <= 6) - P(X <= 2)
pbinom(6, 18, 0.1) - pbinom(2, 18, 0.1)
```

```
# P(X < 4) = P(X <= 3)
pbinom(3, 18, 0.1)

# EXAMPLE 3-28 Bit Transmission Errors: Binomial Mean and Variance

# EXERCISES FOR § 3-8
```

```
# § 3-9 Poisson Process

# Figure 3-33 In a Poission process, events occur at random in an interval.

# § 3-9.1 Poisson Distribution

# EXAMPLE 3-29 Limit of Bit Errors

# EXAMPLE 3-30 Flaws along a Wire

# Figure 3-34 Poisson distribution for selected values of the parameter
# lambda.

# --- 1. 3-9.1 Poisson Distribution & Normal Approximation ---
# Parameters to compare
lambdas <- c(0.1, 2, 5)
u7_max <- max(lambdas + 3 * sqrt(lambdas))
x_seq <- seq(-0.5, floor(u7_max) + 0.5, length.out = 1001)
x_discrete <- 0:floor(u7_max)

# Base Plot (Normal curves)
plot(NULL, xlim = range(x_seq), ylim = c(0, 1.25),
     main = "Poisson(λ) vs. N(λ, λ)",
     xlab = "x", ylab = "f(x)")

colors_p <- c("blue", "red", "darkgreen")
colors_n <- c("lightblue", "orange", "lightgreen")
```

```
for(i in seq_along(lambdas)) {
  # Add Normal Density Lines
  lines(x_seq, dnorm(x_seq, lambdas[i], sqrt(lambdas[i])),
        col = colors_n[i], lwd = 2)
  # Add Poisson PMF Points and Vertical Lines
  points(x_discrete, dpois(x_discrete, lambdas[i]), col = colors_p[i],
         pch = 16)
  segments(x_discrete, 0, x_discrete, dpois(x_discrete, lambdas[i]),
           col = colors_p[i])
}

legend("topright", legend = paste("λ =", lambdas), col = colors_p,
       pch = 16, bty = "n")

# --- 2. Poisson Examples ---

# EXAMPLE 3-31 Probability for Flaws along a Wire

# EXAMPLE 3-31: Flaws along a Wire (λ = 2.3 per mm)
dpois(2, 2.3)                # P(X = 2)
dpois(10, 5 * 2.3)           # P(X = 10 in 5mm)
ppois(0, 2 * 2.3, lower = F) # P(X >= 1 in 2mm)

# EXAMPLE 3-32 Contamination on Optical Disks

# EXAMPLE 3-32: Optical Disks (λ = 0.1 per cm², area = 100)
lambda_disk <- 100 * 0.1
ppois(12, lambda_disk)      # Cumulative P(X <= 12)

# Cumulative Distribution Function

# § 3-9.2 Exponential Distribution

# Figure 3-35 Probability density function of an exponential random
# variable for selected values of lambda.
```

```
# --- 3. 3-9.2 Exponential Distribution ---
# Plotting different rates
rates <- c(2, 0.5, 0.1)
x_exp <- seq(0, 15, length.out = 1000)

plot(NULL, xlim = c(0, 15), ylim = c(0, 2),
     main = "Exponential PDF Comparison",
     xlab = "x", ylab = "f(x)")

for(i in seq_along(rates)) {
  lines(x_exp, dexp(x_exp, rates[i]), col = i, lwd = 2)
}
legend("topright", legend = paste("rate =", rates), col = 1:3, lwd = 2)

# --- EXAMPLE 3-33: Network Log-ons ( λ = 25 per hour) ---
# P(X > 6 min) -> 6 min = 0.1 hours
1 - pexp(0.1, 25)

# P(2 min < X < 3 min)
pexp(3/60, 25) - pexp(2/60, 25)

# 90th Percentile (Time such that 90% of intervals are longer than this)
# Note: "10% of intervals are shorter"
qexp(0.1, 25)

# Visualizing EXAMPLE 3-33
x_val <- seq(0, 0.2, length.out = 500)
plot(x_val, dexp(x_val, 25), type = "l",
     main = "Exponential(25) Distribution", xlab = "x", ylab = "f(x)")
# Shade the area for X > 0.1
x_shade <- seq(0.1, 0.2, length.out = 100)
polygon(c(0.1, x_shade, 0.2), c(0, dexp(x_shade, 25), 0), col = "tomato")

# EXERCISES FOR § 3-9
```

```
# § 3-10 Normal Approximation to the Binomial and Poisson Distributions

# Binomial Distribution Approximation

# Figure 3-37 Normal approximation to the binomial distribution.

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(ggplot2)

# Parameters for Figure 3-37
n <- 10
p <- 0.5
mu <- n * p
sigma <- sqrt(n * p * (1 - p))

# Plotting the Approximation
x_vals <- seq(-0.5, 10.5, length.out = 1001)
y_norm <- dnorm(x_vals, mean = mu, sd = sigma)

plot(x_vals, y_norm, type = "l", col = "red", lwd = 2,
     main = "Binomial(10; 0.5) vs. N(5, 2.5)",
     xlab = "x", ylab = "Density / Probability")

# Add Binomial PMF (The 'h' type creates the vertical lines/bars)
points(0:n, dbinom(0:n, n, p), type = "h", col = "blue", lwd = 2)
points(0:n, dbinom(0:n, n, p), pch = 16, col = "blue")

# Quick verification of P(X = 5) using continuity correction
# Binomial vs. Normal area between 4.5 and 5.5
cat("Binomial P(X = 5):", dbinom(5, 10, 0.5), "\n")
cat("Normal Approx: ", pnorm(5.5, mu, sigma) - pnorm(4.5, mu, sigma), "\n")

# EXAMPLE 3-34 Bit Transmission Errors: Large Sample Size
```

```
# --- 2. EXAMPLE 3-34: Bit Transmission Errors ---
n_34 <- 1.6e7
p_34 <- 1e-5
mu_34 <- n_34 * p_34
sd_34 <- sqrt(n_34 * p_34 * (1 - p_34))

# P(X > 150)
exact_34  <- 1 - pbinom(150, n_34, p_34)
approx_34 <- 1 - pnorm(150.5, mu_34, sd_34)
# Using 150.5 for continuity correction

cat("EXAMPLE 3-34 Exact:", exact_34, "| Approx:", approx_34, "\n")

# EXAMPLE 3-35 Bit Transmission Errors: Normal Approximation

# --- 3. EXAMPLE 3-35: Normal Approximation (Small p) ---
n_35 <- 50
p_35 <- 1/10
mu_35 <- n_35 * p_35
sd_35 <- sqrt(n_35 * p_35 * (1 - p_35))

# P(X <= 2)
exact_35  <- pbinom(2, n_35, p_35)
approx_35 <- pnorm(2.5, mu_35, sd_35)

cat("EXAMPLE 3-35 Exact:", exact_35, "| Approx:", approx_35, "\n")

# Figure 3-38 Binomial distribution is not symmetrical if p is near 0 or 1.

# Poisson Distribution Approximation

# EXAMPLE 3-36 Water Contaminants

# --- 4. EXAMPLE 3-36: Water Contaminants ---
# First part
lambda_36.1 <- 1000
```

```
# P(X <= 949)
exact_36.1  <- ppois(949, lambda_36.1)
approx_36.1 <- pnorm(949.5, lambda_36.1, sqrt(lambda_36.1))

cat("EXAMPLE 3-36 (First part) Exact:", exact_36.1, "| Approx:",
    approx_36.1, "\n")

# Second part
lambda_36.2 <- 1000 * 20 * 10^-3 # lambda = 20
# P(X > 25)
exact_36.2  <- 1 - ppois(25, lambda_36.2)
approx_36.2 <- 1 - pnorm(25.5, lambda_36.2, sqrt(lambda_36.2))

cat("EXAMPLE 3-36 (Second part) Exact:", exact_36.2, "| Approx:",
    approx_36.2, "\n")

# EXERCISES FOR § 3-10
```

```
# § 3-11 More than One Random Variable and Independence

# § 3-11.1 Joint Distributions

# Figure 3-40 Joint probability density function of X and Y.

# Figure 3-41 Probability of a region is the volume enclosed by f(x, y)
# over the region.

# § 3-11.2 Independence

# EXAMPLE 3-37 Optical Drive Diameters

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(tidyverse)

# --- 2. EXAMPLE 3-37: Optical Drive Diameters ---
# Goal: Probability that 10 independent drives all meet specifications.
target_mu <- 0.2508
target_sd <- 0.0005
lower_spec <- 0.2485
upper_spec <- 0.2515
n_drives   <- 10

# Probability that a single drive is within spec
p_single <- pnorm(upper_spec, target_mu, target_sd) -
            pnorm(lower_spec, target_mu, target_sd)

# Joint probability for 10 independent drives
p_all_10 <- p_single^n_drives
print(paste("EXAMPLE 3-37 Probability:", round(p_all_10, 6)))

# EXAMPLE 3-38 Coating Thickness
```

```
# --- 3. EXAMPLE 3-38: Coating Thickness ---
# Goal: Joint probability that three independent dimensions are within
# spec.
# Specs: (Lower, Upper, Mean, SD)
specs <- list(
  layer1 = c(9200, 10800, 10000, 250),
  layer2 = c(950, 1050, 1000, 20),
  layer3 = c(75, 85, 80, 4)
)

# Calculate probabilities for each layer
layer_probs <- sapply(specs, function(s) {
  pnorm(s[2], s[3], s[4]) - pnorm(s[1], s[3], s[4])
})

# Final joint probability (Product of independent events)
p_total_coating <- prod(layer_probs)
print(paste("EXAMPLE 3-38 Probability:", round(p_total_coating, 6)))

# EXAMPLE 3-39 Series System

# P(A ∩ B) = P(A) * P(B)

# EXAMPLE 3-40 Parallel System

# 1 - [P(Aᶜ) * P(Bᶜ)]

# EXAMPLE 3-41 Complex System

# EXERCISES FOR § 3-11
```

```
# § 3-12 Functions of Random Variables

# § 3-12.1 Linear Functions of Independent Random Variables

# EXAMPLE 3-42 Perimeter of a Molded Part

# EXAMPLE 3-43 Perimeter of a Molded Part: Normal Distribution

# EXAMPLE 3-42 & 3-43: Perimeter of a Molded Part
# E(X) = 14, Var(X) = 0.2
# Find P(X > 14.5)
mean_x <- 14
sd_x   <- sqrt(0.2)

# Method 1: Using raw values
prob_1 <- 1 - pnorm(14.5, mean = mean_x, sd = sd_x)

# Method 2: Using Z-score calculation
z_score <- (14.5 - mean_x) / sd_x
prob_2  <- 1 - pnorm(z_score)

cat("Example 3-43 Probabilities:", prob_1, "and", prob_2, "\n")

# § 3-12.2 Linear Functions of Random Variables That Are Not Independent

# Requires covariance/correlation; usually manual calculation or matrix
# algebra

# § 3-12.3 Nonlinear Functions of Independent Random Variables

# EXAMPLE 3-44 Power in a Circuit
```

```
# EXAMPLE 3-44: Power in a Circuit (P = I^2 * R)
# Use the Delta Method or Simulations for nonlinear functions
set.seed(123)
n_sim <- 1000000
I <- rnorm(n_sim, mean = 20, sd = 0.1) # Example values
R <- 80
Power <- (I^2) * R
cat("Approximate Mean Power:", mean(Power), "\n")
cat("Approximate Variance of Power:", var(Power), "\n")
cat("Approximate Standard Deviation of Power:", sd(Power), "\n")

# EXAMPLE 3-45 Resistances in Parallel

# EXAMPLE 3-45: Resistances in Parallel (1/R_total = 1/R1 + 1/R2)
R1 <- rnorm(n_sim, 20, sqrt(0.5))
R2 <- rnorm(n_sim, 50, 1)
R_total <- 1 / (1/R1 + 1/R2)
cat("Approximate Mean Parallel Resistance:", mean(R_total), "\n")
cat("Approximate Variance of Parallel Resistance:", var(R_total), "\n")
cat("Approximate Standard Deviation of Parallel Resistance:", sd(R_total),
    "\n")

# EXERCISES FOR § 3-12

# EXERCISE 3-181 A U-Shaped Assembly

# Let width W = X1 + X2 + X3 - Y
# E(W) = E(X1) + E(X2) + E(X3) - E(Y)
# Var(W) = Var(X1) + Var(X2) + Var(X3) + Var(Y)  <- Note the '+' for
# Variance!
```

```
# § 3-13 Random Samples, Statistics, and the Central Limit Theorem

# --- 1. Setup & Packages ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(tidyverse)

# EXAMPLE 3-46 Strength of O-Rings

# EXAMPLE 3-47 Fill Volumes

# --- EXAMPLE 3-47: Fill Volumes ---
# Goal: Find P(X_bar < 12) given Mean = 12.1, SD = 0.05, n = 10
mu_fill <- 12.1
sd_fill <- 0.05
n_fill  <- 10
se_fill <- sd_fill / sqrt(n_fill) # Standard Error

# Calculation (Both methods yield the same result)
pnorm(12, mean = mu_fill, sd = se_fill)

# Figure 3-43 Distributions of average scores from throwing dice. [Adapted
# with permission from Box, Hunter, and Hunter (1978).]

# EXAMPLE 3-48 Average Resistance

# Figure 3-44 Probability density function of average resistance.

# --- EXAMPLE 3-48: Average Resistance ---
# Goal: Find P(X_bar < 95) given Mean=100, SD=10, n=25
mu_res  <- 100
sd_res  <- 10
n_res   <- 25
se_res  <- sd_res / sqrt(n_res) # Standard Error = 10 / 5 = 2

# Probability Calculation
prob_95 <- pnorm(95, mean = mu_res, sd = se_res)
print(paste("Probability (X_bar < 95):", round(prob_95, 4)))
```

```
# --- Visualizing the Distribution (Figure 3-44) ---
# Define the range (3 standard errors from the mean)
x_vals <- seq(mu_res - 4*se_res, mu_res + 4*se_res, length = 1000)
y_vals <- dnorm(x_vals, mean = mu_res, sd = se_res)

# Create the plot
plot(x_vals, y_vals, type = "l", lwd = 2,
     main = expression(paste("Sampling Distribution of ", bar(X),
     " ~ N(100, ", 2^2, ")")),
     xlab = expression(bar(x)), ylab = "Density", col = "blue")

# Shade the area for P(X_bar < 95)
x_shade <- seq(mu_res - 4*se_res, 95, length = 200)
y_shade <- dnorm(x_shade, mean = mu_res, sd = se_res)
polygon(c(x_shade, 95, mu_res - 4*se_res),
        c(y_shade, 0, 0),
        col = rgb(0, 1, 0, 0.3), border = NA)

# Add a vertical line at the threshold
abline(v = 95, col = "red", lty = 2)

# EXERCISES FOR § 3-13
```